Politecnico di Torino

Master of Science (M.Sc.) in Computer Engineering

Academic Year 2025/2026

Graduation Session March 2026

Design and Implementation of a Cloud-Native Reactive Architecture

The Smart Travel Platform Case Study

Supervisor

Prof. Giovanni Malnati

Candidate

Federico Palazzi

Abstract

The rapid evolution of enterprise web applications has driven a paradigm shift from monolithic architectures to cloud-native *microservices* solutions. This transition is motivated by the need for greater scalability, resilience, and agility in the software development life cycle. This thesis presents the design, development and deployment of "*Smart Travel*", a comprehensive e-commerce platform dedicated to the sale of travel packages. The project serves as a proof of concept (PoC) developed within an industrial internship context, aiming to analyze the complexities and benefits of a modern and distributed software architecture.

The primary objective of this work is not merely the implementation of functional requirements (such as catalog management and order processing), but rather the critical evaluation of competing technologies and the automation of the release process. A central element of this research is the comparative analysis between two leading Java ecosystems: the established *Spring Boot* framework and the cloud-native *Quarkus* framework. Specifically, the project adopts a fully reactive approach for both of them, utilizing *Spring WebFlux* and *Quarkus Reactive* capabilities.

From an architectural perspective, the Smart Travel system is designed to ensure loose coupling and high availability. The backend adopts a microservices pattern supported by *RabbitMQ*, a message broker that facilitates asynchronous *event-driven* communication among services. Data persistence is managed through *MongoDB*, a NoSQL database chosen for its schema flexibility, which is particularly advantageous for handling diverse and evolving travel package structures compared to rigid relational models. On the client side, the user interface is delivered via a *Single Page Application (SPA)* developed with *Angular* and leveraging *RxJS* for reactive state management, ensuring a responsive and fluid user experience.

A significant portion of this thesis is dedicated to the "industrialization" of the software, bridging the gap between development and operations (*DevOps*). The project moves beyond local execution by implementing a robust *Continuous Integration and Continuous Delivery (CI/CD)* pipeline using *Jenkins*. This pipeline automates the entire delivery chain, including code checkout, compilation, container image creation and pushing to a private registry. The final deployment target is *OKD* (the community distribution of *Kubernetes* that powers *Red Hat OpenShift*). The thesis details the orchestration of containerized applications within the OKD cluster, managing critical aspects such as service discovery, route exposure and configuration management via `ConfigMaps` and `Secrets`.

In conclusion, the Smart Travel project demonstrates the viability and robustness of a heterogeneous, fully reactive microservices architecture. It successfully integrates different modern technologies into a cohesive ecosystem, proving that the adoption

of cloud-native principles, while introducing complexity in orchestration and consistency management, offers superior long-term benefits in terms of maintainability and scalability. The work provides a practical blueprint for modernizing legacy Java applications, offering concrete evidence of how the synergy between Reactive frameworks, Event-Driven patterns, and automated DevOps practices defines the state of the art in contemporary software engineering.

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Table of Contents

List of Figures VII

Glossary IX

1 Introduction 1
- 1.1 Context . . . 1
- 1.2 Goal . . . 2
  - 1.2.1 Project description . . . 2
  - 1.2.2 Expected Results . . . 3
- 1.3 Thesis structure . . . 4

2 Background and Technologies 5
- 2.1 Microservices Architecture . . . 5
  - 2.1.1 Principles of Microservices . . . 6
  - 2.1.2 Benefits . . . 7
  - 2.1.3 Challenges and Limitations . . . 7
- 2.2 Imperative vs. Reactive paradigm . . . 8
  - 2.2.1 The Imperative Model . . . 8
  - 2.2.2 The Reactive Model . . . 9
- 2.3 Event-Driven Architectures . . . 10
  - 2.3.1 Synchronous vs. Asynchronous Communication . . . 11
  - 2.3.2 The Role of Message Brokers . . . 11
  - 2.3.3 Benefits and Challenges . . . 12
- 2.4 Backend Technologies . . . 13
  - 2.4.1 Spring Boot: The Industry Standard . . . 13
  - 2.4.2 Quarkus: The Cloud-Native Framework . . . 13
  - 2.4.3 Programming Models and Data Access Paradigms . . . 14
  - 2.4.4 The Rationale for Comparison . . . 15
- 2.5 Frontend Technologies . . . 15
  - 2.5.1 Single Page Application Architecture . . . 16
  - 2.5.2 The Angular Framework . . . 17

2.5.3 Frontend Reactive Programming 18
2.6 Database Technologies 19
2.6.1 NoSQL Concepts and Classifications 19
2.6.2 MongoDB Architecture: Collections vs. Tables 20
2.6.3 Data Processing and Advanced Features 21
2.6.4 Scalability and High Availability 22
2.7 API Paradigms: REST vs. GraphQL 22
2.7.1 REST (Representational State Transfer) 23
2.7.2 GraphQL: The Client-Driven Alternative 25
2.7.3 Choosing the Right Paradigm 27
2.8 DevOps and CI/CD Pipelines 27
2.8.1 The DevOps Philosophy 28
2.8.2 Continuous Integration and Continuous Deployment (CI/CD) 28
2.8.3 The Necessity of Automation at Scale 29

3 System Architecture 30
3.1 Application Domain Overview 30
3.1.1 User Roles and Objectives 31
3.2 Requirements Analysis 32
3.2.1 Functional Requirements 32
3.2.2 Non-Functional Requirements 34
3.3 High-Level Architecture 34
3.3.1 The Backend-For-Frontend Pattern 35
3.3.2 The Microservices Ecosystem 37
3.3.3 Event-Driven Payment and Notification Flow 37
3.4 Data Modeling 38
3.5 Communication Patterns 38

4 Application Design and Implementation 39

5 Infrastructure Automation and Cloud Deployment 40

6 Conclusion 41

Bibliography 42

List of Figures

2.1 Monolithic vs. Microservices architecture 6
2.2 Thread-per-Request model in an imperative application. 8
2.3 Reactive model in a non-blocking application. 9
2.4 Synchronous vs. Asynchronous communication 11
2.5 Event-Driven Architecture 12
2.6 Angular Architecture 18
2.7 NoSQL Database Types 20
2.8 SQL vs. MongoDB concepts 21
2.9 REST vs. GraphQL 25
2.10 DevOps 28
2.11 CI/CD pipeline 29

3.1 Smart Travel High-Level Architecture 35
3.2 Backend-For-Frontend pattern 36

Glossary

ACID

Atomicity, Consistency, Isolation, Durability

AJAX

Asynchronous JavaScript and XML

API

Application Programming Interface

BFF

Backend-For-Frontend

BSON

Binary JSON

CDC

Change Data Capture

CI/CD

Continuous Integration / Continuous Delivery

DBMS

Database Management System

DI

Dependency Injection

DOM

Document Object Model

DX

Developer Experience

EDA

Event Driven Architecture

JSON

JavaScript Object Notation

MPA

Multi-Page Application

NFR

Non-Functional Requirements

NoSQL

Not Only SQL

ORM

Object-Relational Mapping

PoC

Proof of Concept

POJO

Plain Old Java Object

Pub/Sub

Publish/Subscribe

RBAC

Role Based Access Control

RDBMS

Relational Database Management System

RxJS

Reactive Extensions for JavaScript

SDL

Schema Definition Language

SEO

Search Engine Optimization

SPA

Single Page Application

UI

User Interface

URL

Uniform Resource Locator

Chapter 1

Introduction

1.1 Context

The rapid evolution of enterprise web applications over the last decade has driven a significant paradigm shift in software engineering, moving away from traditional monolithic architectures towards cloud-native microservices solutions.

Historically, monolithic applications were the industry standard and they integrated user interface, business logic, and data access layers into a single deployable unit. They offered simplicity during the early stages of development, straightforward local testing, and centralized debugging. However, as applications grow in size, complexity, and user base, the inherent limitations of the monolithic approach become increasingly apparent [1].

A monolithic design relies on tightly coupled components. Consequently, a minor update, a new feature addition, or a simple bug fix requires the entire massive codebase to be recompiled, tested, and redeployed. This rigid structure often leads to development bottlenecks and significantly increases the risk of system-wide failures during deployments. Furthermore, scaling a monolith is inherently inefficient; if a single specific component (such as the payment processing module) experiences a surge in traffic, the entire monolithic application must be scaled horizontally, leading to unnecessary resource consumption and inflated operational costs.

To overcome these scalability and maintainability challenges, the industry has widely adopted the microservices architecture [1]. This modern paradigm decomposes a large application into a suite of small, autonomous, and loosely coupled services. Each microservice is organized around a specific business capability, encapsulates its own data, and communicates with other services through lightweight, well-defined application programming interfaces (APIs) or asynchronous message brokers. This decentralized approach offers superior modularity, improved fault isolation - the

failure of one service does not inevitably crash the entire system - and the agility to deploy and scale individual services independently based on their specific demand. However, the transition to microservices is not without its own set of complexities. It introduces challenges such as increased operational overhead, the need for sophisticated monitoring and logging solutions, and the complexity of managing distributed transactions and data consistency across services. Despite these challenges, the microservices architecture has become the de facto standard for building scalable, resilient, and maintainable enterprise applications in the cloud era.

1.2 Goal

The primary objective of this thesis is to explore, apply, and critically evaluate modern cloud-native technologies and microservices architectures. To achieve this, the "*Smart Travel*" platform was designed and developed as a proof of concept (PoC). Rather than simply focusing on the implementation of functional business requirements, this project serves primarily as a practical testbed to analyze the complexities, challenges, and benefits of building a modern, distributed software system. The project was conducted during an internship at a software company, situated directly within the industry's broader context of architectural modernization.

1.2.1 Project description

The Smart Travel application was designed as a comprehensive e-commerce platform dedicated to the sale of travel packages. The system manages the entire customer journey, from the public consultation of a catalog containing flights, accommodations, and organized activities, to the execution of the purchasing process, order history management, and the automated dispatch of email notifications.
A central technical goal of this research is the critical evaluation and comparative analysis of two leading Java ecosystems used to build the platform's microservices: the established *Spring Boot* framework and the newer cloud-native *Quarkus* framework. Specifically, the project adopts a fully reactive programming paradigm for both frameworks, utilizing *Spring WebFlux* and *Quarkus Reactive* capabilities. This enables non-blocking, asynchronous execution, allowing the system to handle a massive number of concurrent requests with a highly optimized `thread pool`.
Furthermore, this thesis aims to explore the "industrialization" of software through the adoption of *DevOps* practices. The objective is to move beyond local execution by implementing a robust *Continuous Integration and Continuous Delivery* (CI/CD) pipeline. By automating the entire delivery chain - from code checkout and compilation to container image creation and pushing to a private registry - the project demonstrates how to bridge the gap between development and operations. Finally, the thesis details the deployment and orchestration of these containerized

microservices within an *OKD* cluster (the community distribution of *Red Hat OpenShift*), managing critical operational aspects such as service discovery, route exposure, and configuration management.

1.2.2 Expected Results

The execution of this thesis work is expected to produce both concrete technical artifacts and valuable analytical insights. The expected results can be categorized into three main areas:

1. From a software engineering perspective, the project aims to deliver a fully functional PoC of the e-commerce platform. This includes the successful implementation of the frontend interface and the backend microservices, validating the practical application of asynchronous communication via RabbitMQ and flexible data management using MongoDB. The resulting system is expected to demonstrate the resilience and scalability inherent in a properly designed cloud-native architecture.

2. From an analytical standpoint, the research provides a qualitative comparative evaluation between Spring Boot and Quarkus within a reactive programming context. Given the scope of the internship, the comparison focuses heavily on the *Developer Experience* (DX) rather than quantitative runtime metrics. The expected outcome is a clear assessment of the differences in syntax, configuration management, boilerplate code reduction, and the overall ease of implementing reactive paradigms in both frameworks. This comparison will offer valuable insights into the practical development lifecycle, ecosystem maturity, and the learning curve associated with adopting Quarkus compared to the traditional Spring environment.

3. From an operational perspective, the expected result is the successful demonstration of a fully automated DevOps lifecycle. By delivering a working CI/CD pipeline via Jenkins that culminates in a live deployment on an OKD cluster, the project will prove the viability and necessity of infrastructure automation. This includes the ability to manage containerized applications at scale, handle configuration and secrets management effectively, and ensure seamless service discovery and routing in a production-like environment.

1.3 Thesis structure

The thesis is structured as follows:

- **Chapter 2: Background and Technologies** - Provides an overview of the core concepts and technologies underpinning the project, including microservices and event-driven architectures, imperative and reactive programming paradigms, backend and frontend frameworks (Spring Boot, Quarkus, Angular), NoSQL databases (MongoDB), and API communication models (REST vs. GraphQL).
- **Chapter 3: System Architecture** - Details the architectural decisions, technology stack, and design patterns used in the Smart Travel project. It covers the microservices decomposition, the event-driven communication via message brokers, and the frontend design.
- **Chapter 4: Application Design and Implementation** - Offers a detailed description of the development process. This includes data modeling, API contract definitions, and explanations of key software components, accompanied by code snippets comparing the Spring Boot and Quarkus implementations.
- **Chapter 5: Infrastructure Automation and Cloud Deployment** - Describes the different steps of the CI/CD pipeline, Jenkins automation configurations, and the final orchestration of containerized applications within the OKD cluster environment.
- **Chapter 6: Conclusion** - Summarizes the project findings, evaluates the results of the technology comparison, discusses the limitations encountered during development, and outlines future work directions for the platform.

Chapter 2

Background and Technologies

This chapter provides a comprehensive overview of the core concepts and technologies underpinning the project. It begins by exploring the architectural shift from traditional monolithic systems to decentralized microservices, then moving to the main differences between imperative and reactive programming models, highlighting how the latter addresses the high-concurrency demands of modern cloud infrastructure.
Furthermore, it examines the specific technological stack selected for the application, including the frontend framework, backend environments, and NoSQL database management. It details the different communication paradigms adopted — spanning from synchronous REST and GraphQL APIs to asynchronous Event-Driven Architectures — to allow the system's isolated components to seamlessly interact with each other.
Finally, it introduces the core principles of DevOps and Continuous Integration/Continuous Deployment pipelines, establishing the necessity of automated infrastructure for managing and deploying modern microservices at scale.

2.1 Microservices Architecture

The evolution of cloud computing has fundamentally altered how enterprise applications are designed, developed, and maintained. While monolithic architectures served as the industry standard for decades, they present significant scaling and maintenance limitations as systems grow in complexity. To address these bottlenecks, the software industry has increasingly adopted the microservices architecture, a paradigm that decomposes large, rigid applications into a suite of smaller, independent services [2].

2.1.1 Principles of Microservices

Unlike a monolithic application that shares a single massive codebase and a centralized database, a microservices architecture is built upon the principle of decentralization and loose coupling. Each microservice is designed around a specific business capability or "bounded context" [3]. For example, in an e-commerce platform like Smart Travel, the catalog management and the order processing capabilities are handled by distinct autonomous services.
The foundational principles of this architecture include:

- **Autonomy and Independent Deployment:** Each service is a self-contained unit. It encapsulates its own code, dependencies, and typically its own data store. This allows teams to update, deploy, and scale a single service without needing to recompile or redeploy the entire system.

- **Lightweight Communication:** Because microservices operate as independent processes, they must communicate over a network to fulfill complex business transactions. This is achieved using well-defined *Application Programming Interfaces* (APIs), typically relying on synchronous protocols like HTTP or asynchronous message delivery mechanisms.

- **Decentralized Data Management:** Rather than relying on a single shared database schema, each microservice manages its own database, choosing the DBMS technology best suited to its specific data access patterns.

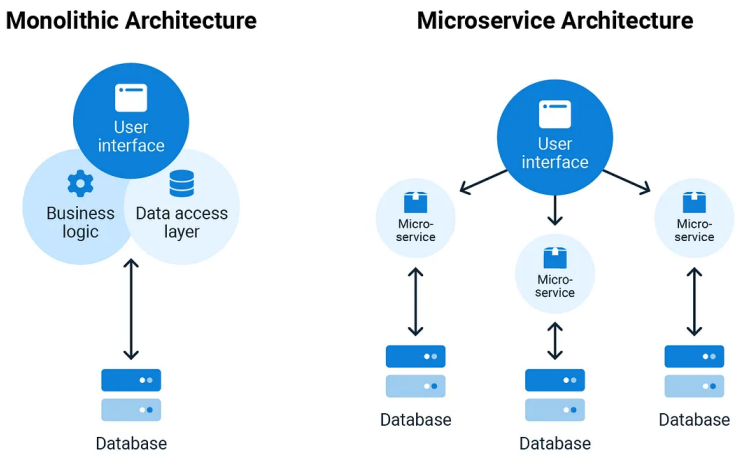

Figure 2.1: Monolithic vs. Microservices architecture. *Source: [4]*

2.1.2 Benefits

The shift from monolithic to microservices architecture provides several advantages [5]:

- **Targeted Scalability:** In a monolithic system, an unexpected spike in traffic for one specific feature (e.g., the payment gateway) requires scaling the entire application, leading to resource wastage. Microservices allow infrastructure teams to scale only the specific services experiencing high demand, optimizing cloud resource consumption and costs.
- **Fault Isolation and Resilience:** Because microservices are loosely coupled, a critical bug, memory leak, or crash in one service does not inherently cascade and bring down the entire platform. The application can handle partial failures gracefully, degrading functionality while keeping core operations online.
- **Technological Freedom:** Teams are not locked into a single technology stack. Since microservices communicate via standardized network protocols, developers can select the most appropriate programming language, framework, and database for each specific task. This flexibility allowed the Smart Travel project to implement and compare different Java frameworks (Spring Boot and Quarkus) within the same ecosystem.
- **Agility and CI/CD:** Smaller codebases are easier to understand, test, and deploy. This structure naturally complements DevOps practices, enabling CI/CD pipelines to push frequent updates to production.

2.1.3 Challenges and Limitations

Despite its significant advantages, adopting a microservices architecture introduces a high degree of distributed system complexity. Some of the key challenges include:

- **Operational Overhead and Monitoring:** Managing dozens of independently running services is inherently more complex than managing a single monolith. It requires sophisticated container orchestration platforms (such as Kubernetes or OKD) and centralized observability tools for distributed tracing, log aggregation, and metric monitoring to identify where a failure occurred across the network.
- **Network Latency and Congestion:** Replacing in-memory function calls with remote network requests introduces latency. If a single user action requires a long chain of inter-service API calls, the accumulated network overhead can severely degrade system performance.

- **Data Consistency:** Because each service maintains its own database, achieving data consistency across the system becomes a major challenge. Traditional `ACID` transactions that span multiple tables are no longer feasible. Instead, developers must rely on complex distributed transaction patterns and embrace *eventual consistency*.

2.2 Imperative vs. Reactive paradigm

The choice of programming paradigm fundamentally dictates how an application manages system resources, handles concurrency, and scales under load. In the context of modern Java microservices, understanding the underlying thread management of both approaches is crucial for evaluating their suitability for high-concurrency, I/O-bound applications.

2.2.1 The Imperative Model

Traditional applications use blocking I/O and an imperative (sequential) execution model. In a web application context - such as a traditional Spring MVC application - this paradigm relies on the *Thread-per-Request* concurrency model [6].
When a new HTTP request arrives, the server allocates a dedicated thread from a pre-configured thread pool to handle the entire lifecycle of that request. If the application needs to interact with an external resource, such as querying a database or calling a third-party API, the thread initiates the I/O operation and then completely halts execution (blocks) until it receives a response.
While this model is straightforward to write, debug, and reason about, it is highly inefficient for I/O-bound microservices. During the waiting period, the blocked thread consumes valuable memory and CPU cycles via context switching, doing absolutely no useful work. Under sudden spikes in traffic, the thread pool can quickly become exhausted. Once all threads are blocked waiting for slow external services, the application becomes unresponsive to new user requests, severely limiting scalability.

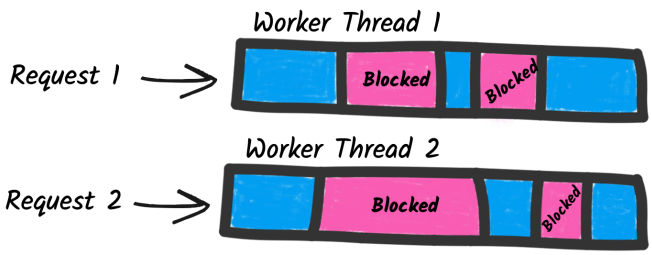

Figure 2.2: Thread-per-Request model in an imperative application. *Source: [7]*

2.2.2 The Reactive Model

The reactive model was designed to build non-blocking applications that are highly responsive, resilient, and resource-efficient. Instead of the Thread-per-Request model, reactive frameworks utilize an *Event Loop* architecture. In this model, a very small number of threads known as `event loop threads` continuously loop to process incoming requests.
When an I/O operation is required, the thread does not block. Instead, it delegates the task to a non-blocking I/O handler, registers a callback, and immediately returns to the event queue to process the next incoming HTTP request. Once the external database or API finishes its processing, it pushes an event back to the event loop, which then resumes the original request and sends the response to the client [7].

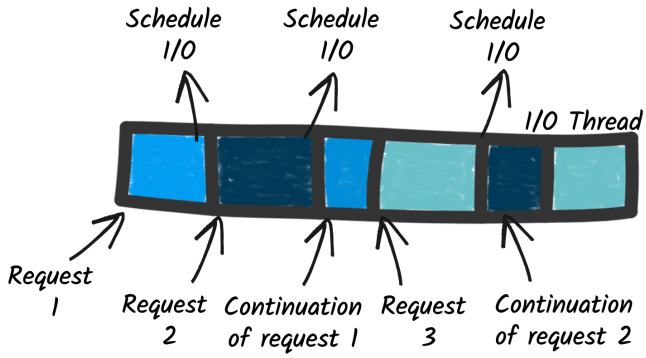

Figure 2.3: Reactive model in a non-blocking application. *Source: [7]*

The key advantages of the reactive model include:

- **Resource Efficiency:** By eliminating blocked threads, the system can handle thousands of concurrent connections using a fraction of the memory and CPU.

- **Data as Streams:** Data is not pulled sequentially but pushed asynchronously. Frameworks like Spring WebFlux and Quarkus Mutiny provide rich APIs to compose and transform these data streams without blocking the main execution flow.

The primary advantage of the reactive concurrency model is its resource efficiency and scalability. By ensuring that threads are never blocked waiting for external I/O operations, the system can sustain thousands of concurrent connections using a fraction of the memory and CPU required by a traditional imperative system. This architectural shift makes the reactive model particularly well-suited for I/O-bound

microservices, allowing them to remain highly responsive and maintain stable performance even under massive, sudden surges in network traffic.

However, this model introduces significant complexities regarding development and operational debugging. The most critical is the inclusion of a blocking operation — such as a legacy JDBC database query, a synchronous HTTP call, or a heavy CPU-bound computation — directly on the event loop. Because the system relies on a small pool of threads, blocking even a single one can immediately bottleneck the application. Furthermore, the non-linear execution flow fragments traditional stack traces. When an exception occurs, the stack trace often only reflects the point of failure within the asynchronous callback, losing the historical context of the original user request and making root-cause analysis considerably more difficult than in the linear imperative model.

2.3 Event-Driven Architectures

As microservices ecosystems expand, the method by which these independent services communicate becomes a critical architectural decision. While traditional web architectures rely heavily on synchronous, point-to-point interactions, the inherent limitations of this approach at scale have led to the widespread adoption of *Event-Driven Architectures* (EDA) [8].

In a tightly coupled microservices environment, relying solely on direct HTTP/REST calls can quickly result in a fragile "distributed monolith." As the number of services grows, point-to-point communication creates a complex, tangled web of dependencies. If a single core service experiences downtime or network latency, the effects can cascade throughout the dependency chain, potentially bringing down the entire platform. Furthermore, this approach introduces strict *temporal coupling*: both the calling service and the receiving service must be online, available, and responsive at the exact same moment for a transaction to succeed.

To mitigate these risks, EDA introduces a fundamental paradigm shift. Instead of services directly commanding one another to perform actions, they broadcast "events" — immutable records indicating that a significant change in state or a specific business action has already occurred (for example, a `PaymentProcessed` or `UserRegistered` event). Other autonomous services within the ecosystem can then asynchronously listen for and react to these events at their own pace. This decoupling transforms the architecture from a rigid, command-driven model into a highly reactive system, vastly improving overall resilience and allowing individual components to scale independently based on event volume.

2.3.1 Synchronous vs. Asynchronous Communication

In a synchronous communication model, typically implemented via HTTP and REST APIs, a service sends a request to another service and blocks its own execution until it receives a direct response. While conceptually simple and easy to trace, this tightly coupled approach creates cascading bottlenecks. If the receiving service is slow, overwhelmed, or offline, the calling service is forced to wait, consuming resources and potentially causing system-wide performance degradation [9].
On the other hand, asynchronous communication decouples the sender from the receiver. In an Event-Driven Architecture, services communicate by producing and consuming *events* - lightweight messages that represent a significant change in state or a specific occurrence within the business domain (e.g., "user registered" or "order placed"). The service producing the event simply broadcasts it and immediately resumes its own operations, entirely unaware of who will consume the event or when it will be processed [8].

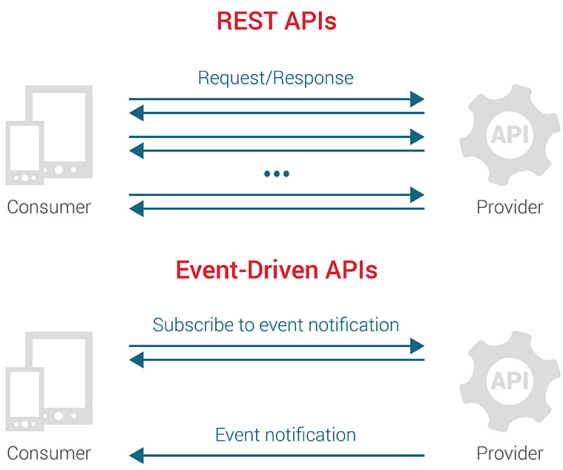

Figure 2.4: Synchronous vs. Asynchronous communication. *Source: [10]*

2.3.2 The Role of Message Brokers

To facilitate this asynchronous exchange without requiring services to know each other's network locations or IP addresses, EDA relies on a centralized intermediary known as a message broker [11].
This architecture generally relies on three primary components:

- **Producers (Publishers):** The microservices that generate events when a specific business action occurs. Their only responsibility is to format the event data and safely push it to the broker.

- **Message Brokers:** The middleware infrastructure that receives, stores, and routes the events. The broker ensures reliable message delivery, managing queues or topics so that events are safely held even if the intended downstream recipients are currently offline or busy.

- **Consumers (Subscribers):** The microservices that continuously listen to the broker for specific types of events. Upon receiving an event, the consumer processes the data and executes its own independent business logic.

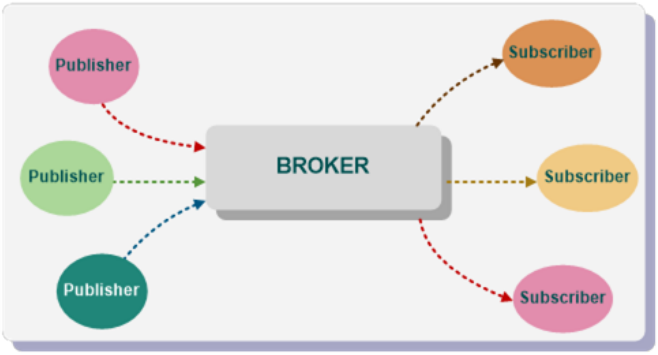

Figure 2.5: Event-Driven Architecture

This publish/subscribe (Pub/Sub) model allows multiple different consumers to react to a single event simultaneously and independently.

2.3.3 Benefits and Challenges

Implementing an Event-Driven Architecture provides different advantages. The decoupling of services enables exceptional scalability, as individual components can be scaled up or down based on their specific event processing loads. Furthermore, it significantly enhances overall system resilience: if a consumer service crashes, the message broker simply retains the pending events in a queue until the service is restored, guaranteeing no data loss during the outage [9].
However, this paradigm introduces unique complexities regarding distributed data. Because communication is fundamentally asynchronous, achieving immediate transactional data consistency across the entire platform is impossible. Instead, developers must design applications around the concept of *eventual consistency*, where the system state reconciles over time. Additionally, tracking the flow of a single user action across multiple asynchronously triggered microservices requires sophisticated distributed tracing mechanisms and centralized logging [11].

2.4 Backend Technologies

Java has been the cornerstone of enterprise backend development for decades, favored for its robustness, mature ecosystem, and platform independence. However, as the industry transitioned from monolithic architectures running on heavy application servers to containerized microservices orchestrated by Kubernetes, traditional Java frameworks faced significant challenges. They often suffered from slow startup times and high memory footprints, characteristics misaligned with the rapid scaling and high-density deployment needs of cloud-native environments [12].
To address these challenges, the Java ecosystem has evolved significantly. During the development of this thesis' project, two distinct frameworks were selected to build the backend microservices: the established Spring Boot and the modern cloud-native Quarkus.

2.4.1 Spring Boot: The Industry Standard

Introduced by Pivotal Software (now VMware Tanzu) in 2014, Spring Boot revolutionized Java enterprise development. It was designed to simplify the bootstrapping and development of new Spring applications by adhering to the *convention over configuration* principle. Before Spring Boot, developers spent hours configuring XML files and managing complex application server deployments. Spring Boot eliminated this friction by providing auto-configuration, opinionated dependency management, and embedded web servers (such as Tomcat or Netty), allowing developers to package an entire microservice into a single, easily deployable `fat JAR` [13].
Today, Spring Boot is the de facto standard for building Java microservices. It can count on an enormous ecosystem, with out-of-the-box integrations for almost every conceivable database, message broker, and cloud provider. While its traditional roots lie in the synchronous, imperative programming model (*Spring MVC*), the framework has actively modernized. With the introduction of *Spring WebFlux*, it now fully supports the reactive, event-loop programming paradigm discussed in Section 2.2, enabling high concurrency without the overhead of thread-per-request blocking.

2.4.2 Quarkus: The Cloud-Native Framework

While Spring Boot adapted to the cloud era, Quarkus was born entirely within it. Created by Red Hat and released in 2019, Quarkus describes itself as a "Kubernetes Native Java framework tailored for GraalVM and HotSpot". Its primary mission is to optimize Java for containerized and serverless environments, where fast startup

times (measured in milliseconds) and minimal memory consumption are critical for reducing cloud infrastructure costs and enabling rapid auto-scaling [12].
Quarkus achieves this extreme efficiency by fundamentally changing the Java build process. Traditional frameworks like Spring heavily rely on reflection, dynamic class loading, and classpath scanning at runtime (processes that consume significant memory and slow down the boot time). Quarkus, instead, utilizes a *compile-time boot* technique. It shifts the vast majority of framework initialization and configuration parsing to the build phase (*Ahead-Of-Time* processing).
Furthermore, Quarkus is built from the ground up on a reactive core powered by Eclipse Vert.x [14]. This unified architecture allows developers to seamlessly blend standard imperative code and non-blocking reactive code within the exact same application, dynamically optimizing the underlying execution thread model without requiring the developer to switch between entirely different API stacks.

2.4.3 Programming Models and Data Access Paradigms

At their core, both Spring Boot and Quarkus serve the exact same fundamental purpose: they are comprehensive Java frameworks designed to build robust, production-ready backend web applications and microservices. They abstract away the complex infrastructure required for enterprise software — such as exposing HTTP endpoints, managing database connection pools, and orchestrating dependency injection—allowing developers to focus strictly on domain-specific business logic.
However, while their end goals are identical, they take fundamentally different approaches to their internal programming models and data access paradigms.
From an API and dependency injection perspective, Spring Boot relies heavily on its own proprietary, highly mature ecosystem. Developers utilize Spring-specific annotations (such as `@RestController`, `@Autowired`, and `@Service`) to wire the application together. Conversely, Quarkus was designed to leverage existing industry standards. It relies on MicroProfile and Jakarta EE specifications, utilizing CDI (Contexts and Dependency Injection) with `@Inject` for bean management and JAX-RS (Java API for RESTful Web Services) with annotations like `@Path` and `@GET` for defining network endpoints.
The most striking architectural difference, however, lies in how the two frameworks handle data access and domain modeling:

- **Spring Data and the Repository Pattern:** Spring Boot strictly encourages the *Repository Pattern*. In this model, the domain entity (the class representing a database document or row) is kept as a pure POJO (Plain Old Java Object). Data access logic is strictly separated into a distinct interface (e.g., `ReactiveMongoRepository`), which Spring automatically implements at

runtime. This provides a clean separation of concerns but requires creating multiple files and interfaces even for simple CRUD operations [15].

- **Quarkus Panache and the Active Record Pattern:** To drastically reduce boilerplate code, Quarkus introduces a layer called *Panache* (available for both Hibernate ORM and MongoDB). While Panache supports the traditional Repository pattern, its standout feature is its implementation of the *Active Record Pattern*. In this paradigm, the domain entity itself extends a base class (such as `PanacheMongoEntity`) and directly inherits static data access methods. This allows developers to query or save data directly through the entity class (e.g., `Flight.findById(id)` or `flight.persist()`). This tight coupling of data and behavior significantly accelerates development for straightforward business domains, though it can become difficult to manage in highly complex relational schemas [16].

2.4.4 The Rationale for Comparison

The decision to utilize both Spring Boot and Quarkus within the Smart Travel platform was driven by the desire to evaluate the state of the art in Java microservices. Implementing identical business logic - such as reactive database queries and asynchronous RabbitMQ event publishing/consuming - using both frameworks provides a valuable lens through which to assess their respective developer experiences, syntactic clarity, and ease of configuration. While Spring Boot offers the safety and familiarity of a massive, mature ecosystem, Quarkus promises a leaner, faster, and easier approach to cloud-native development [17].

2.5 Frontend Technologies

Just as backend architectures have evolved to handle greater scale and complexity, frontend development has undergone a massive transformation. The traditional web paradigm, where the server renders and delivers completely new HTML pages for every user interaction, has largely been superseded by the *Single Page Application* (SPA) model. This shift was driven by the need for more dynamic, responsive, and app-like user experiences that could not be achieved with full-page reloads. To implement this modern frontend architecture, the *Angular* framework was selected for the project, leveraging its powerful component-based structure and native support for reactive programming with the *RxJS* library.

2.5.1 Single Page Application Architecture

Historically, web applications were built as Multi-Page Applications (MPAs). In an MPA, every time a user clicks a link or submits a form, the browser discards the current page, sends a request to the server, waits for the server to process the logic, and then downloads an entirely new HTML document to render. While this approach is straightforward and naturally optimized for *Search Engine Optimization* (SEO), it is highly inefficient for complex interactive applications. The constant full-page reloads lead to latency, wasted bandwidth, and a disjointed user experience [18].

A Single Page Application flips this paradigm. As the name suggests, an SPA loads only a single foundational HTML web page upon the user's initial visit. This initial payload includes all the necessary static assets - such as HTML scaffolding, CSS stylesheets, and the core JavaScript application bundle.

Once the application is loaded into the browser, the page is never reloaded again during the user's session. Instead, when the user interacts with the application (e.g., searching for a flight or filtering hotel results), JavaScript intercepts the action and dynamically rewrites the current DOM (*Document Object Model*). If new data is required, the SPA makes asynchronous requests (via `AJAX` or the `Fetch API`) to backend APIs. The server responds with lightweight JSON data rather than heavy HTML markup. The frontend then processes this data and injects it directly into the existing page structure [18].

This architecture provides several significant advantages:

- **Fluid User Experience:** By eliminating the disruptive "white flash" of page reloads, SPAs provide a continuous, fluid experience that closely mimics a native desktop or mobile application.

- **High Performance and Speed:** After the initial load, subsequent interactions are nearly instantaneous because only raw data is transmitted over the network, drastically reducing bandwidth consumption and server load.

- **Clear Separation of Concerns:** The SPA architecture perfectly complements a microservices backend. The frontend becomes a completely independent application that only cares about consuming data via standardized APIs, allowing frontend and backend teams to develop, deploy, and scale their codebases independently.

Despite its benefits, the SPA model introduces challenges, particularly regarding the initial loading time - since the browser must download a large JavaScript bundle upfront - and the complexity of managing client-side routing and browser history. To address these complexities, robust frontend frameworks are required.

2.5.2 The Angular Framework

To implement the SPA architecture for the Smart Travel platform, Angular was selected. Maintained by Google, Angular is a comprehensive, enterprise-grade framework specifically designed for building scalable single-page applications [19]. Unlike lightweight libraries that focus solely on rendering views (such as React), Angular provides a highly opinionated, all-encompassing toolset. It enforces a strict architectural pattern, requiring developers to write code using TypeScript — a statically typed superset of JavaScript that introduces interfaces, classes, and advanced object-oriented features. This ensures greater code reliability, earlier error detection during compilation, and improved maintainability in large-scale projects.
At its core, the Angular framework is built upon several fundamental concepts that work together to create a cohesive architecture:

- **Modules:** Angular applications are inherently modular. An `NgModule` is a container that groups related components, directives, pipes, and services into cohesive blocks of functionality. Every application has at least one root module (typically `AppModule`) that bootstraps the application. Enterprise applications often utilize numerous "feature modules" that allow for *lazy-loading* — loading specific parts of the application only when the user navigates to them, thereby significantly reducing the initial JavaScript payload.

- **Components and Templates:** The UI is divided into encapsulated, reusable building blocks called components. Each component strictly separates its concerns: a TypeScript class manages the state and behavior, an HTML template defines the view, and a CSS/SCSS file scopes styling exclusively to that component. The connection between the class and the template is managed through powerful *data binding* mechanisms (such as string interpolation, property binding, and event binding), which automatically synchronize the UI with the underlying application state.

- **Directives:** While components define custom DOM elements, directives are classes that add additional behavior to elements in the Angular application. They are primarily divided into *structural directives*, which alter the layout of the DOM by adding, replacing, or removing elements (such as `*ngIf` for conditional rendering and `*ngFor` for list iteration), and *attribute directives*, which alter the appearance or behavior of an existing element (such as dynamically applying CSS classes with `ngClass`).

- **Services and Dependency Injection (DI):** To ensure components remain lean and focused solely on presenting data, business logic and data fetching are delegated to *Services*. Angular features a robust, hierarchical *Dependency*

Injection (DI) system. Instead of a component manually instantiating a class to fetch data, it declares a dependency in its constructor. The DI framework then automatically provides a singleton instance of the requested service—such as an HTTP client used to query the catalog microservice. This pattern promotes loosely coupled, highly modular, and easily testable code [19].

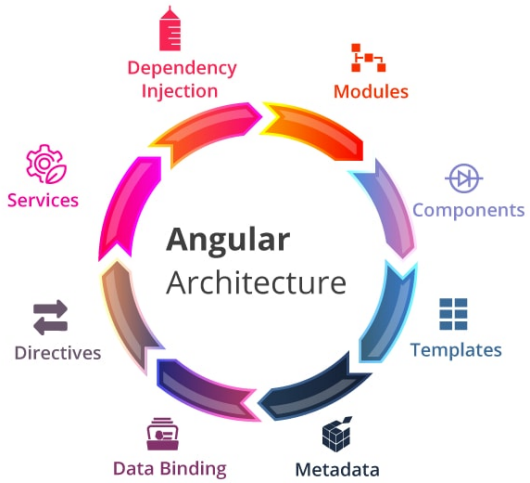

Figure 2.6: Angular Architecture. *Source: [20]*

2.5.3 Frontend Reactive Programming

Because SPAs heavily rely on asynchronous events — such as user inputs, delayed API responses, and routing changes — managing this complex flow of data is a major challenge. To handle this, Angular natively integrates the RxJS library.
RxJS brings the reactive programming paradigm (discussed in Section 2.2) directly into the browser. It introduces the concept of *Observables*, which represent continuous streams of asynchronous data or events over time [21].
Instead of relying on traditional JavaScript `Promises`, which can only handle a single future value and cannot be cancelled, Observables can emit multiple values over time and can be closed or cancelled at any moment. For example, when performing a search query, the Angular HTTP client returns an Observable. Using RxJS operators (like `map`, `filter`, or `debounceTime`), the frontend can gracefully transform the incoming data, ignore rapid repeated clicks from the user, and seamlessly update the UI state without resorting to convoluted, heavily nested callback functions [21]. By leveraging RxJS, the frontend achieves the same non-blocking, event-driven resilience as the reactive backend microservices.

2.6 Database Technologies

As modern applications demand greater flexibility, performance, and the ability to handle massive volumes of unstructured or semi-structured data, traditional *Relational Database Management Systems* (RDBMS) often face significant limitations. The rigid, tabular schema of SQL databases requires careful upfront planning and costly migrations when data requirements change. Furthermore, scaling a relational database is typically restricted to vertical scaling (adding more CPU or RAM to a single server), which eventually hits a hardware ceiling [22]. To address these challenges, *NoSQL* databases emerged as a powerful alternative, offering distributed architectures and highly adaptable data models.

2.6.1 NoSQL Concepts and Classifications

NoSQL databases depart from the fixed table-and-row model, prioritizing horizontal scalability, high availability, and agility over strict ACID compliance. Instead of forcing data into rigid relational structures, NoSQL systems allow data to be stored in formats that more closely align with the objects used in modern application code [23].

NoSQL databases are generally categorized into four primary models:

- **Key-Value Stores:** The simplest NoSQL model, where data is stored as a collection of unique keys and associated values (e.g., Redis). They are highly optimized for fast lookups and caching.

- **Wide-Column Stores:** Data is organized into tables, rows, and dynamic columns, allowing each row to have different columns. They excel at handling massive, predictable query workloads (e.g., Cassandra).

- **Graph Databases:** Designed to store and traverse relationships between entities using nodes and edges (e.g., Neo4j). They are ideal for social networks, fraud detection, or recommendation engines.

- **Document Databases:** Data is stored in flexible, JSON-like documents. Each document contains pairs of fields and values, which can include nested arrays and objects. This is the category to which MongoDB belongs [22].

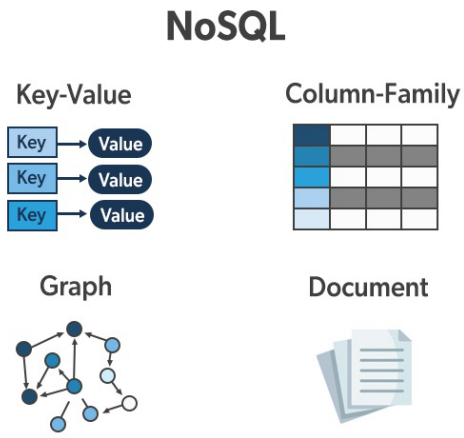

Figure 2.7: NoSQL Database Types

2.6.2 MongoDB Architecture: Collections vs. Tables

The Smart Travel application leverages MongoDB as its primary data store. MongoDB is the leading NoSQL document database, natively designed to handle modern application data with superior flexibility and scalability. Its fundamental departure from traditional relational databases lies in how it structures and stores records.
In a traditional RDBMS, data is structured into *tables* consisting of *columns* (attributes) and *rows* (records). Every row in a table must adhere strictly to the predefined columns. Representing complex, hierarchical data — such as a travel order containing multiple passengers, nested payment details, and a list of booked activities — requires "normalizing" the data. This means splitting the information across multiple distinct tables and reassembling it at query time using computationally expensive `JOIN` operations.
On the other hand, MongoDB organizes data into *Collections* (the conceptual equivalent of tables) and *Documents* (the equivalent of rows).

- **Documents:** A document is a self-contained unit of data. Under the hood, MongoDB stores these documents in a binary format called `BSON` (Binary JSON), which extends standard JSON with additional data types like dates, decimals, and raw binary data. Unlike a flat SQL row, a BSON document can store rich, hierarchical data structures, including nested sub-documents and arrays. This means an entire "Order" can be retrieved in a single, highly efficient database read operation. Furthermore, this document-oriented architecture natively maps to the object-oriented structures used in Java or TypeScript, largely eliminating the need for complex *Object-Relational Mapping* (ORM) layers [23].

- **Collections:** Documents are grouped together into Collections. However, unlike SQL tables, collections enforce a *flexible schema* (or dynamic schema). In a relational database, adding a new feature — such as introducing a "discount_code" or "dietary_requirements" field to only a subset of travel packages — requires an expensive `ALTER TABLE` migration that can temporarily lock the database and force null values upon all existing rows. In MongoDB, documents within the same collection do not need to have identical structures. A new field can simply be appended to a new document without affecting existing records.

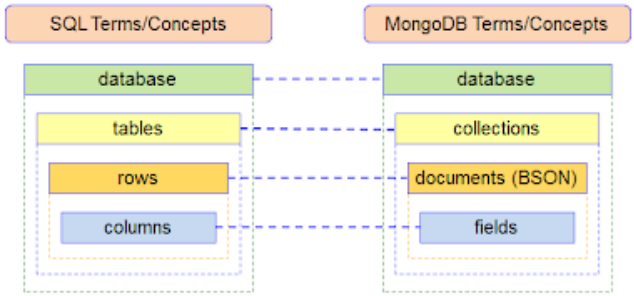

Figure 2.8: SQL vs. MongoDB concepts. *Source: [24]*

This schema flexibility significantly accelerates the development lifecycle and perfectly accommodates the diverse, constantly evolving attributes of different travel products within the Smart Travel catalog.

2.6.3 Data Processing and Advanced Features

MongoDB provides a highly capable and expressive query ecosystem to manipulate and retrieve data efficiently:

- **Aggregation Framework:** To perform complex data processing operations — such as filtering, sorting, grouping, and calculating totals — MongoDB utilizes the Aggregation Pipeline. Modeled on the concept of data processing pipelines in Unix, documents enter a multi-stage pipeline where each stage transforms the data as it passes through. Stages like `$match` (filtering), `$group` (aggregating data), and `$project` (reshaping the output document) allow developers to perform advanced analytics directly within the database engine without needing to transfer massive datasets to the application layer [25].

- **Indexing Capabilities:** Despite its flexible schema, MongoDB relies heavily on indexes to ensure high query performance. Like traditional RDBMS, it uses `B-tree` structures to prevent the database from performing full collection scans. It supports secondary indexes, compound indexes, and highly specialized index types such as geospatial indexes (crucial for location-based travel searches) and text-search indexes.
- **ACID Transactions:** Historically, a major criticism of NoSQL databases was their lack of transactional guarantees across multiple records. However, since version 4.0, MongoDB has introduced full support for multi-document ACID transactions across replica sets and sharded clusters. This allows developers to execute complex, multi-step operations (such as processing a payment and simultaneously updating an order status) with the guarantee that either all changes are applied successfully, or none are, bridging the reliability gap between relational and document databases [26].

2.6.4 Scalability and High Availability

Beyond its data model, MongoDB's architecture is built for distributed environments:

- **Replica Sets:** To ensure high availability, MongoDB uses Replica Sets — a cluster of nodes (typically one Primary and multiple Secondaries) that maintain identical copies of the data. If the Primary node fails, the system automatically elects a new Primary, providing seamless failover and redundancy [23].
- **Sharding:** When the data volume exceeds the capacity of a single machine, MongoDB handles horizontal scaling through Sharding. The data is partitioned across multiple servers based on a designated "Shard Key," allowing the database to manage massive datasets without degrading query performance.

While MongoDB offers immense freedom, it still requires careful data modeling. Developers must strategically choose between *embedding* related data (storing it within the same document for faster read operations) and *referencing* it (linking separate documents via IDs to avoid duplicating large amounts of data). This decision is crucial for optimizing query performance and ensuring data integrity, especially in a complex domain like travel where entities such as flights, hotels, and user profiles have intricate relationships.

2.7 API Paradigms: REST vs. GraphQL

In a decoupled architecture where the frontend is separated from the backend microservices, the API serves as the fundamental communication bridge. The

design of these APIs dictates how efficiently data is transferred over the network. For the "Smart Travel" platform, evaluating the right API paradigm was essential for optimizing the interaction between the Angular frontend and the Java backend. The two dominant paradigms in modern web development are *REST* and *GraphQL*.

2.7.1 REST (Representational State Transfer)

REST has been the undisputed standard for web APIs for over a decade. Introduced by Roy Fielding in 2000, it is an architectural style rather than a strict protocol. RESTful APIs are built around the concept of "resources" (e.g., users, flights, orders), which are identified by standard URLs. Clients interact with these resources using standard HTTP methods such as `GET` (to retrieve data), `POST` (to create data), `PUT` (to update), and `DELETE` to remove data [27].

Core Principles of REST

To be considered truly RESTful, an application's API must adhere to several guiding architectural constraints:

- **Client-Server Separation:** The user interface concerns (the client) are strictly separated from the data storage and business logic concerns (the server). This decoupling allows the frontend (e.g., the Angular SPA) and the backend microservices to evolve and scale independently.
- **Statelessness:** Each client request to the server must contain all the information necessary to understand and process it. The server does not store any client context or session state between requests. This stateless nature makes REST highly scalable, as any server replica within a cluster can handle any incoming request without needing to synchronize session data.
- **Cacheability:** Responses must explicitly define themselves as cacheable or non-cacheable. If a response is cacheable, the client or a network intermediary can reuse the data for equivalent later requests, significantly improving network efficiency and reducing server load.
- **Layered System:** A client cannot ordinarily tell whether it is connected directly to the end server or to an intermediary (like a load balancer, an API Gateway, or a proxy). This enables load balancing and shared caches without altering the client's behavior.
- **Uniform Interface:** This is the fundamental feature that distinguishes a REST API. It simplifies the architecture by enforcing a standard, uniform way to interact with the system, relying heavily on standard HTTP specifications.

Resources and Representations

In REST, the primary abstraction of information is a *resource*. A resource can be any concept, entity, or collection within the business domain — such as a user, a flight, or an order. Resources are uniquely identified by standard URLs, often referred to as endpoints [27]. For example, the endpoint `/flights` might represent the collection of all flight resources, while `/flights/123` would represent a specific flight with the ID of `123`.
It is important to note that when a client requests a resource, the server does not send the actual internal database record; instead, it sends a *representation* of the resource's state at that specific moment. In modern web development, this representation is almost exclusively formatted in lightweight JSON objects.
Clients manipulate these resources using standard HTTP verbs, which map directly to traditional database CRUD (Create, Read, Update, Delete) operations:

- `GET` (Read): Retrieves a representation of the resource. It is a safe and idempotent operation, meaning it does not alter the state of the server.
- `POST` (Create): Submits a new payload to the server to create a new subordinate resource.
- `PUT` (Update): Updates an entire existing resource with a new representation, or creates it if it does not exist.
- `DELETE` (Delete): Removes the specified resource from the server.

Limitations of REST in Modern SPAs

While REST's statelessness and uniform interface make it the default choice for general-purpose microservice-to-microservice communication, it has revealed significant limitations regarding data fetching as frontend applications have grown more complex:

- **Over-fetching:** A REST endpoint typically returns a fixed data structure. If the Smart Travel frontend only needs the names and prices of available flights to display a simple list, a `GET /flights` request might still return massive JSON objects containing seating charts, airline details, and baggage rules that the client must download over the network but ultimately discard [28].
- **Under-fetching (the N+1 problem):** If a view requires data from multiple related resources—such as a user's profile and their past order history—the client is often forced to make multiple sequential HTTP requests to different endpoints (e.g., `GET /users/123` followed by `GET /users/123/orders`). This significantly increases network latency and degrades the overall user experience.

2.7.2 GraphQL: The Client-Driven Alternative

To solve the inefficiencies of over-fetching and under-fetching, Facebook developed GraphQL, open-sourcing it in 2015. Unlike REST, which exposes multiple URLs for different resources, a GraphQL API typically exposes only a single endpoint (e.g., `/graphql`) [29].
Instead of the server dictating the structure of the response, GraphQL transfers control to the client. It is a strictly typed query language that allows the frontend to explicitly ask for exactly the data it needs, and nothing more. The client sends a specific query document in the body of an HTTP `POST` request, and the server responds with a JSON object that exactly mirrors the shape of the requested query. For example, a single GraphQL query can simultaneously fetch a user's name, their last three orders, and the specific hotel names associated with those orders. The backend GraphQL engine acts as an orchestrator, resolving the disparate data points from various underlying databases or microservices and assembling them into a single, highly optimized response [28].

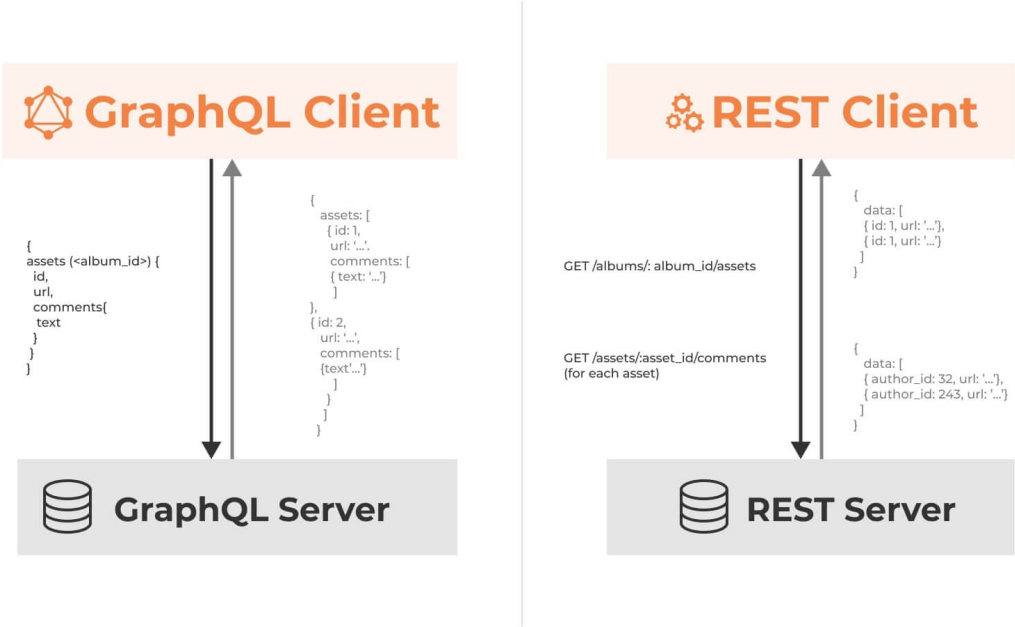

Figure 2.9: REST vs. GraphQL

Schema and Type System

At the core of every GraphQL server is a strictly defined contract known as the *Schema*, written in the *Schema Definition Language* (SDL). The schema defines every piece of data that can be queried or modified, explicitly declaring the types of objects, their fields, and the relationships between them [29].

At the foundational level of this type system are the primitive data types, known in GraphQL as *Scalars*. Scalars represent the leaves of any query — they are the concrete data values that resolve to a single point and cannot be queried any further. GraphQL provides five default scalar types out of the box:

- `Int`: A signed 32-bit integer.
- `Float`: A signed double-precision floating-point value.
- `String`: A UTF-8 character sequence.
- `Boolean`: A standard `true` or `false` value.
- `ID`: A unique identifier, typically used to refetch an object or as a key for a cache. While it is serialized as a String, defining a field as an `ID` signifies that it is not intended to be human-readable.

Beyond these defaults, the GraphQL specification allows developers to define *Custom Scalars*. This is particularly helpful for platforms like Smart Travel, where domain-specific data types such as "Flight", "Accommodation", or "Activity" can be represented as custom scalars with their own validation rules and serialization logic.
Because GraphQL is strongly typed, the server validates every incoming client request against this schema before attempting to execute it. This guarantees that the client only asks for data that actually exists and provides a massive boost to the developer experience, as frontend tooling can automatically generate code and provide real-time auto-completion based on the schema's exact structure.

Core Operations: Queries, Mutations, and Subscriptions

While REST relies on HTTP methods to define the intent of an operation, GraphQL handles all requests via `HTTP POST` and defines intent using three distinct operation types:

- **Query:** The equivalent of a REST `GET` request. Queries are used by the client to fetch data without causing any side effects on the server. A major feature of a Query is that the structure of the returned JSON data perfectly mirrors the nested structure of the requested query. For example, a single Query can simultaneously fetch a user's name, their last three orders, and the specific hotel names associated with those orders, entirely avoiding the N+1 request problem.
- **Mutation:** The equivalent of REST `POST`, `PUT`, and `DELETE` requests. Mutations are used to write data to the server — such as creating a new user,

booking a flight, or cancelling an order. In GraphQL, a Mutation not only performs the intended write operation but can also return the newly modified object in the exact same request, allowing the frontend application to instantly update its local state.

- **Subscription:** While Queries and Mutations follow a standard request-response cycle, Subscriptions establish a persistent, long-lasting connection between the client and the server, typically using WebSockets. Subscriptions are used to push real-time updates from the server to the client whenever a specific event occurs (e.g., pushing a notification when a flight's price suddenly drops).

By relying on a single endpoint, a strongly typed schema, and flexible operations, GraphQL provides a highly efficient and declarative data-fetching paradigm perfectly suited for modern Single Page Applications.

2.7.3 Choosing the Right Paradigm

While GraphQL offers unparalleled efficiency for frontend data fetching, it introduces significant complexity on the backend. Developing a GraphQL server requires defining a rigorous schema, implementing complex resolver functions, and carefully managing database queries to prevent malicious or overly deep queries from exhausting server resources. Furthermore, traditional HTTP caching mechanisms are much harder to implement with GraphQL than with REST [29].

In the context of modern platforms like Smart Travel, the two paradigms are often not mutually exclusive. REST remains highly effective for server-to-server microservice communication and simple CRUD (Create, Read, Update, Delete) operations, while GraphQL is increasingly favored as an API Gateway layer to serve perfectly tailored data payloads to frontend applications.

2.8 DevOps and CI/CD Pipelines

The architectural transition from monolithic applications to distributed microservices fundamentally changes not only how software is written but also how it is delivered. Managing, testing, and deploying dozens of independent services manually is highly prone to human error and operationally unsustainable. To address this bottleneck, the software industry has widely adopted DevOps practices and automated deployment pipelines [30].

2.8.1 The DevOps Philosophy

Traditionally, software development and IT operations functioned in isolated silos. Developers wrote the code and pass it to operations teams, who were then responsible for deploying and maintaining it. This often led to misaligned incentives: developers wanted to push new features rapidly, while operations teams prioritized system stability, naturally resisting frequent changes [31].
DevOps is a cultural and technical movement that merges these two disciplines (Development and Operations). It promotes a collaborative environment where cross-functional teams take full ownership of the entire application lifecycle — from initial planning and writing code to deploying it into production and monitoring its live performance. The core objective of DevOps is to shorten the development lifecycle, increase deployment frequency, and achieve highly dependable releases [30].

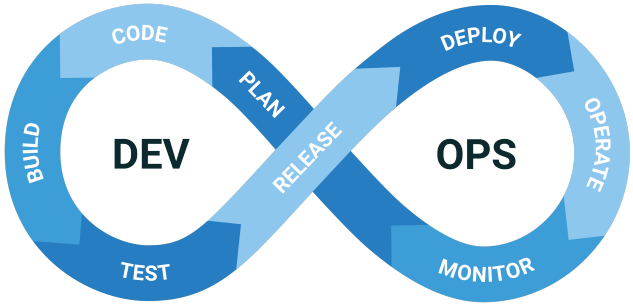

Figure 2.10: DevOps

2.8.2 Continuous Integration and Continuous Deployment (CI/CD)

The technical engine that drives the DevOps philosophy is the CI/CD pipeline. A pipeline is a sequence of automated steps that source code must pass through before it reaches the end user. It enforces strict quality gates and removes manual intervention from the software release process [32].

- **Continuous Integration (CI):** In a microservices environment, multiple developers frequently commit code changes to a shared version control repository (such as Git). CI is the practice of automatically triggering a build process every time a new commit is pushed. The CI server automatically compiles the code and runs a suite of unit and integration tests. This rapid feedback loop ensures that new code seamlessly integrates with the existing codebase

and that bugs are caught immediately, rather than during a massive release at the end of the month.

- **Continuous Delivery and Deployment (CD):** Once the code passes the CI phase, the CD phase automates the release process. In *Continuous Delivery*, the built artifact (such as a Docker container image) is automatically prepared and pushed to a staging environment, awaiting a manual trigger for final production release. In *Continuous Deployment*, even this manual trigger is removed: every change that passes all automated tests is deployed directly to production without human intervention [33].

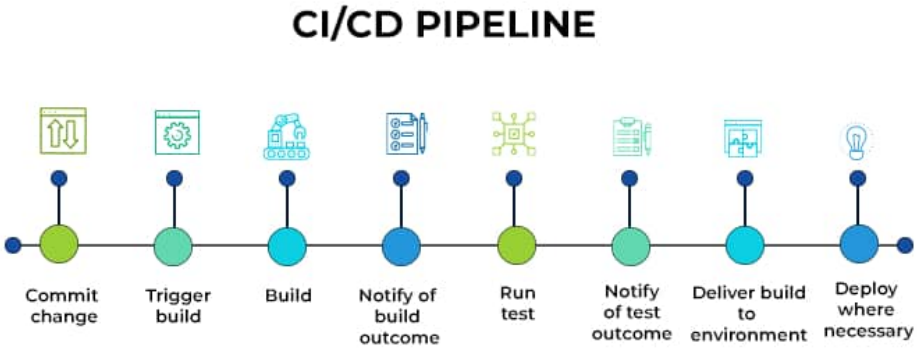

Figure 2.11: CI/CD pipeline

2.8.3 The Necessity of Automation at Scale

Implementing robust CI/CD pipelines is not merely a convenience for cloud-native applications, but a structural necessity. Because a platform like Smart Travel is composed of multiple independent services (e.g., Users, Travel Catalog, Orders, Notification) built with varying technologies (Angular, Spring Boot, Quarkus), each service requires its own distinct lifecycle and pipeline.
Automation guarantees repeatability. By defining the infrastructure and deployment steps as code, a pipeline ensures that a microservice will be compiled, tested, packaged into a container, and deployed in the exact same manner every single time. This eliminates configuration drift, drastically reduces the downtime associated with manual deployments, and provides the agility required to roll back seamlessly if a critical defect is detected in production [32].

Chapter 3

System Architecture

This chapter begins by establishing the business context and user roles of the Smart Travel domain, followed by an analysis of its functional and non-functional requirements. It then explores the high-level architecture, detailing the decomposition of the monolithic domain into bounded, autonomous microservices (Users, Travel Catalog, Orders, and Notifications).
It proceeds to examine the data modeling strategy, highlighting how MongoDB's flexible schema accommodates the diverse structures of travel products.
Finally, the chapter outlines the communication patterns employed to orchestrate these services, contrasting synchronous API gateways with the asynchronous event-driven mechanisms powered by RabbitMQ and Debezium.

3.1 Application Domain Overview

Before diving into the technical requirements and the architectural topology of the system, it is essential to establish the business context and the primary objectives of the application.
Smart Travel was conceived as a full-stack e-commerce application dedicated to the tourism sector. Its primary business goal is to provide a centralized hub where users can search, compose, and purchase complete travel experiences — spanning flights, accommodations, local activities, and travel packages. From a software engineering perspective, the platform serves as a PoC to validate the implementation of cloud-native microservices, reactive programming paradigms, and event-driven communication.
To achieve these goals, the application was designed around a specific set of core business requirements and user-facing functionalities:

- **Public Catalog Exploration:** The platform provides public access to the travel catalog, allowing users to explore available offerings using advanced

search and filtering mechanisms to find specific flights, hotels, or activities.

- **Flexible Purchasing and Cart Management:** The system supports a highly flexible shopping experience. Users can purchase individual travel services standalone or aggregate them into complete and customized packages, with the ability to review and modify their cart selections prior to checkout.

- **End-to-End Order Management:** The platform handles the complete purchasing lifecycle, providing authenticated customers with a dedicated personal area to track and monitor their past and upcoming orders.

- **External Service Integration:** To facilitate real-world e-commerce capabilities, the application securely integrates with external third-party services, such as online payment gateways (e.g., PayPal) to process financial transactions.

- **Automated Notifications:** To ensure a responsive user experience, the system implements an automated notification pipeline, dispatching immediate email confirmations upon the successful completion of an order.

3.1.1 User Roles and Objectives

To satisfy diverse business needs, the platform was designed to support multiple distinct user profiles, each with specific permissions, workflows, and end goals. The system identifies four primary actors:

- **Guest (Unregistered User):** Guests can freely navigate the platform to explore the catalog of travel packages and search for flights, accommodation and activities on specific dates. However, they are restricted from executing financial transactions or composing custom packages until they authenticate.

- **Customer (Registered User):** Once authenticated, a Customer aims to plan and purchase their travel itinerary. Beyond the capabilities of a Guest, Customers can compose personalized travel packages by aggregating distinct catalog components, proceed to secure checkout and manage a personal dashboard containing their order history.

- **Travel Agent:** A business-oriented user responsible for enriching the platform's catalog. The Travel Agent's primary goal is to curate and compose new travel packages. They have access to dedicated interfaces where they can bundle flights, hotels, and activities into fixed "Agency Packages," manage their publication status (draft, published, or archived), and view historical data on the packages they have created.

- **Administrator:** The Administrator possess the highest level of authorization, granting them access to the user management system and the ability to oversee all travel packages (including those created by Travel Agents). The Administrator's role is to maintain the integrity of the platform by managing users' accounts and permissions.

3.2 Requirements Analysis

Before defining the technical architecture, a requirements analysis was conducted to establish the scope and expected behavior of the application. The requirements were divided into functional, detailing what the system must do, and non-functional, dictating how the system should perform under various conditions.

3.2.1 Functional Requirements

The functional requirements were categorized by business domain and mapped to specific user roles (Guest, Customer, Travel Agent, and Administrator). The following table outlines the comprehensive list of functional features expected from the platform.

Table 3.1: Smart Travel Functional Requirements

ID	Description
FR1	**Authentication and Authorization**
FR1.1	User login and logout
FR1.2	Account creation (Sign up)
FR1.3	Account management (Edit details, Delete account)
FR2	**Flights**
FR2.1	Search flights by destination and date range
FR2.2	Filter search results
FR2.3	View flight details (e.g., layovers, airlines)
FR2.4	Flight payment processing (Summary, Total calculation)
FR3	**Accommodations**
FR3.1	Search accommodations by destination and date range
FR3.2	Filter search results
FR3.3	View accommodation details (e.g., available rooms, amenities)
FR3.4	Accommodation payment processing (Summary, Total calculation)
FR4	**Activities**

Continued on next page

Table 3.1 – continued from previous page

ID	Description
FR4.1	Search activities by destination and date range
FR4.2	Filter search results
FR4.3	View activity details (e.g., location, schedule)
FR4.4	Activity payment processing (Summary, Total calculation)
FR5	**Travel Packages (Agency Created)**
FR5.1	Search existing agency packages by destination and date range
FR5.2	Filter search results
FR5.3	View package details (Flights, Accommodation, Activities)
FR5.4	Package payment processing
FR5.5	View recommended travel packages on the homepage
FR6	**Custom Travel Packages (Customer Created)**
FR6.1	Search individual components to build a custom package
FR6.2	Select components and compose the custom package
FR6.3	Custom package payment processing
FR7	**Customer Personal Area**
FR7.1	View order history and specific order details
FR7.2	View and manage account details
FR8	**Notifications**
FR8.1	Send an automated summary email to the customer for every purchase
FR9	**Package Management (Travel Agent)**
FR9.1	Add a new travel package (Draft / Unpublished)
FR9.2	Publish a previously created draft package
FR9.3	Archive a published travel package
FR9.4	Delete an unpublished travel package
FR9.5	View history of created travel packages
FR10	**User Management (Administrator)**
FR10.1	View all user accounts
FR10.2	Edit user accounts details
FR10.3	Delete user accounts

3.2.2 Non-Functional Requirements

While the functional requirements dictate the platform's features, the non-functional requirements (NFR) establish the systemic qualities necessary for the application to succeed in a cloud-native production environment. Based on the project goals, the following constraints were prioritized:

- **Scalability:** The system must gracefully handle varying loads. By adopting a microservices architecture, individual domains (such as the Travel Catalog service during heavy search traffic) must be able to scale horizontally and independently without requiring the entire platform to be duplicated.

- **Performance and Responsiveness:** To provide a fluid user experience, the frontend SPA must minimize full-page reloads. On the backend, services should leverage reactive programming (via Spring WebFlux and Quarkus Reactive) to ensure high throughput and minimize blocking operations, keeping API response times consistently low even under concurrent load.

- **Resilience and Fault Tolerance:** The failure of a single component must not trigger a system-wide outage. The architecture must employ asynchronous communication (Event-Driven messaging via RabbitMQ) to decouple critical write operations — such as processing an order and sending a confirmation email — ensuring that temporary downstream unavailability does not result in data loss.

- **Maintainability and Automation:** The codebase must be highly modular, allowing separate development teams to work on the frontend, catalog, and order services simultaneously. Furthermore, the deployment lifecycle must be entirely automated via CI/CD pipelines to ensure consistent, repeatable, and testable releases into the target OKD cluster.

3.3 High-Level Architecture

To satisfy the scalability, maintainability, and functional requirements described in the previous section, the application was designed utilizing a distributed microservices architecture. The system is composed of several independent backend services, a centralized API gateway, an event broker, and a dedicated frontend application.

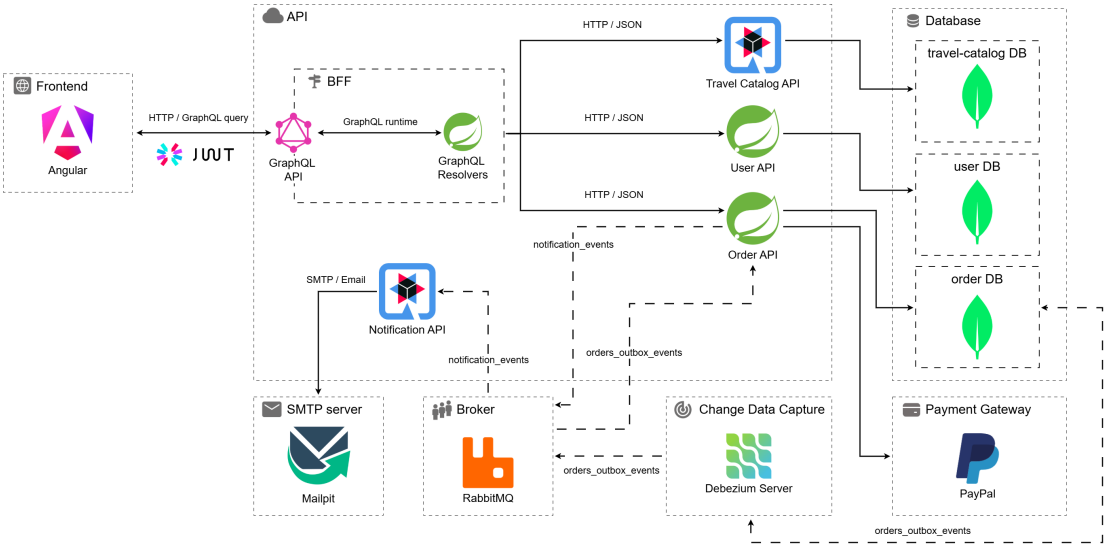

Figure 3.1: Smart Travel High-Level Architecture

3.3.1 The Backend-For-Frontend Pattern

Instead of allowing the Angular frontend to communicate directly with the underlying microservices, the architecture implements the *Backend-For-Frontend* (BFF) pattern.

In a distributed microservices ecosystem, a frontend client often needs to fetch data from multiple independent services to render a single view. Allowing the client to communicate directly with these backend services introduces several critical issues: it creates lots of network traffic with high latency, tightly couples the frontend to the backend schema, and exposes internal network architectures to the public internet [34].

While a traditional API Gateway solves the exposure problem by acting as a single entry point, a single, general-purpose gateway serving all types of clients (e.g., web browsers, mobile apps, third-party consumers) quickly becomes a bloated, monolithic bottleneck [35].

Introduced to solve this exact problem, the BFF pattern advocates for creating a dedicated backend service tailored specifically for the needs of a single frontend interface. In this way, the BFF can implement client-specific logic, such as aggregating data from multiple microservices into a single response, transforming data formats, and enforcing security policies without affecting other clients or services.

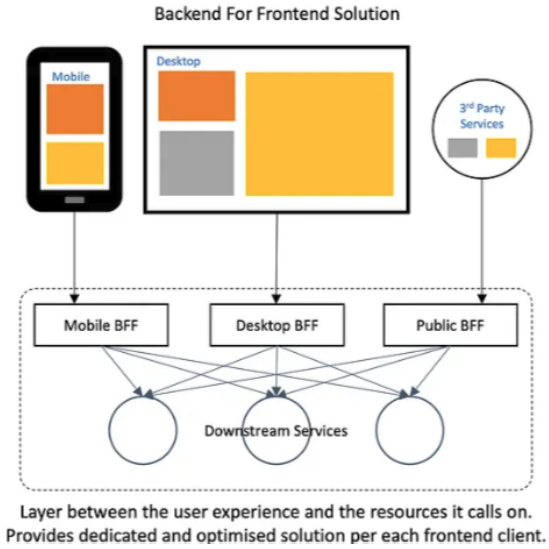

Figure 3.2: Backend-For-Frontend pattern. *Source: [36]*

Implementation in Smart Travel

For the Smart Travel platform, the BFF is tailored to the requirements of the Angular SPA. It fulfills two primary architectural responsibilities:

- **Data Aggregation and GraphQL Translation:** When the frontend sends a GraphQL query, the BFF acts as an orchestrator. Its internal resolvers parse the query, execute the necessary downstream HTTP/JSON requests to the Travel Catalog, User, and Order microservices, aggregate the responses in memory and return a single JSON payload to the client. This translation layer completely eliminates the N+1 request problem and prevents both over-fetching and under-fetching on the client side.

- **Centralized Security Boundary:** The BFF acts as the strict security perimeter for the platform. It handles all authentication and authorization checks using **Spring Security** and **JSON Web Tokens (JWT)**. Because the BFF validates the JWTs and enforces *Role-Based Access Control* (RBAC) before routing any request downstream, the individual microservices do not need to implement complex or redundant security checks. They operate within a trusted internal network, safely assuming that any request forwarded by the BFF has already been authenticated.

3.3.2 The Microservices Ecosystem

The core business logic is divided into four autonomous microservices. Adhering to the "Database-per-Service" pattern, each data-persisting microservice manages its own isolated MongoDB instance hosted on MongoDB Atlas, ensuring true loose coupling. To evaluate the state-of-the-art Java frameworks, these services were developed using a mix of Spring Boot and Quarkus:

- **Travel Catalog API (Quarkus):** Manages the core product domain. It handles searches, filtering, and retrieval of flights, accommodations, activities, and travel packages. It connects to the dedicated `travel-catalog DB`.
- **User API (Spring Boot):** Manages user profiles, account details and system roles. It connects to the `user DB`.
- **Order API (Spring Boot):** Orchestrates the checkout process, order history, and payment validation. It connects to the `order DB`.
- **Notification API (Quarkus):** A purely asynchronous, event-driven service responsible for dispatching communications to the user which does not require a database. Instead, it listens to the message broker and interacts with Mailpit, a fake SMTP server, to send emails.

3.3.3 Event-Driven Payment and Notification Flow

One of the most complex architectural flows in the Smart Travel platform is the payment process, which leverages the *Outbox Pattern* alongside *Change Data Capture* (CDC) to guarantee data consistency across distributed services.
While the granular implementation of this flow will be detailed in subsequent chapters, the high-level orchestration follows these strict steps:

1. **Checkout Initiation:** The frontend requests an order creation via the BFF. The Order API creates a `PENDING` order in MongoDB, retrieves a payment URL from PayPal, and the user is redirected to authorize the transaction.
2. **Payment Authorization:** After the user authorizes the payment, they are redirected back to the application. The Order API updates the order status to `PAID` and, crucially, writes an event to a database outbox collection within the exact same transaction.
3. **Change Data Capture (CDC):** A **Debezium Server** continuously monitors the MongoDB change streams. Upon detecting the new outbox insert, Debezium captures the change and publishes it to the `orders_outbox_events` queue in **RabbitMQ**.

4. **Asynchronous Capture:** The Order API, acting as a consumer, reads the outbox event from RabbitMQ. It officially captures the funds via PayPal, updates the database status to `COMPLETED`, and publishes a new event to the `notification_events` queue.

5. **Notification Dispatch:** The Notification API consumes the event from the `notification_events` queue and triggers an SMTP server (Mailpit) to send a finalized order confirmation email to the customer.

This sophisticated flow ensures that no data is lost even if a service temporarily fails. If the Notification API is down, RabbitMQ simply holds the event in the queue until the service recovers, perfectly demonstrating the resilience of Event-Driven Architecture discussed in Section 2.3.

3.4 Data Modeling

3.3 Data Modeling: Designing the MongoDB collections and handling flexible schemas.

3.5 Communication Patterns

3.4.1 Synchronous communication (REST/GraphQL)
3.4.2 Asynchronous communication (RabbitMQ events, Debezium event streaming, Mailpit as fake SMTP server)

Chapter 4

Application Design and Implementation

4.1 Backend Development: * 4.1.1 Implementing the reactive services with Spring Boot
4.1.2 Implementing the reactive services with Quarkus
4.2 Frontend Development: Building the user interface, state management with RxJS, and connecting to the backend.
4.3 Core Workflows:
4.3.1 The catalog search and filtering mechanism
4.3.2 The payment integration flow
4.3.3 The event-driven notification system

Chapter 5

Infrastructure Automation and Cloud Deployment

5.1 The DevOps Culture: Bridging the gap between development and operations.
5.2 Continuous Integration & Continuous Delivery (CI/CD): Automating the build, test, and containerization process using Jenkins.
5.3 Container Orchestration with Kubernetes/OKD: * 5.3.1 OKD Cluster overview
5.3.2 Workload management: Pods and Deployments
5.3.3 Configuration and Security: ConfigMaps and Secrets
5.3.4 Networking: Services and Route exposure

Chapter 6

Conclusion

6.1 Technology Evaluation: Spring Boot vs. Quarkus: Comparing developer experience, startup times, resource consumption, and reactive capabilities.
6.2 Architectural Evaluation: Assessing the scalability, resilience, and maintainability of the implemented microservices and event-driven patterns.
6.3 Testing and Validation: Ensuring the functional requirements of the Smart Travel application were met.

Bibliography

[1] Muskaan Goyal and Pranav Bhasin. *Moving from monolithic to microservices architecture for multi-agent systems*. 2025. URL: https://arxiv.org/pdf/2505.07838 (cit. on p. 1).

[2] Amazon Web Services (AWS). *What are Microservices?* 2025. URL: https://aws.amazon.com/microservices/ (cit. on p. 5).

[3] Microsoft Learn. *Microservices Architecture Style*. 2025. URL: https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices (cit. on p. 6).

[4] CoderCo. *Monoliths vs Microservices: A Guide to Choosing the Right Architecture for Your Application*. 2023. URL: https://blog.coderco.io/p/monoliths-vs-microservices-a-guide (cit. on p. 6).

[5] Eyitayo Olaigbe. *Microservices Architecture: Building Resilient, Scalable Applications in the Cloud Era*. 2024. URL: https://www.tayoolaigbe.com/articles/microservices-architecture (cit. on p. 7).

[6] Ray Dhiraj. *Spring WebFlux Reactive REST API*. 2023. URL: https://www.devglan.com/spring-boot/spring-webflux-reactive-rest-api (cit. on p. 8).

[7] Quarkus Official Documentation. *Quarkus Reactive Architecture*. 2024. URL: https://quarkus.io/guides/quarkus-reactive-architecture (cit. on pp. 8, 9).

[8] IBM. *What Is Event-Driven Architecture?* 2024. URL: https://www.ibm.com/topics/event-driven-architecture (cit. on pp. 10, 11).

[9] Microsoft Learn. *Asynchronous message-based communication*. 2022. URL: https://learn.microsoft.com/en-us/dotnet/architecture/microservices/architect-microservice-container-applications/asynchronous-message-based-communication (cit. on pp. 11, 12).

[10] Lydia Defranchi. *What are event-driven APIs? Understanding event-driven vs REST API interactions*. 2025. URL: https://blog.axway.com/learning-center/apis/basics/event-driven-vs-rest-api-interactions (cit. on p. 11).

[11] Red Hat. *Event-driven architecture: Understanding the essential benefits*. 2021. URL: https://www.redhat.com/en/blog/event-driven-architecture-essentials (cit. on pp. 11, 12).

[12] Rich Sharples / Red Hat. *Bringing Java into the Kubernetes-native future with Quarkus*. 2020. URL: https://www.redhat.com/en/blog/bringing-java-kubernetes-native-future-quarkus (cit. on pp. 13, 14).

[13] Spring.io. *Microservices with Spring*. 2025. URL: https://spring.io/microservices/ (cit. on p. 13).

[14] Eclipse Vert.x. *Vert.x*. URL: https://vertx.io (cit. on p. 14).

[15] Spring.io. *Spring Data*. 2025. URL: https://spring.io/projects/spring-data (cit. on p. 15).

[16] Quarkus Official Documentation. *Simplified MongoDB with Panache*. 2025. URL: https://quarkus.io/guides/mongodb-panache (cit. on p. 15).

[17] Hitesh Umaletiya / Brilworks. *Spring Boot vs Quarkus vs Micronaut Performance and Use Case Guide*. 2025. URL: https://www.brilworks.com/blog/spring-boot-vs-quarkus-vs-micronaut/ (cit. on p. 15).

[18] Adobe Experience Cloud Team. *Single-page applications (SPAs) — what they are and how they work*. 2023. URL: https://business.adobe.com/blog/basics/learn-the-benefits-of-single-page-apps-spa (cit. on p. 16).

[19] Angular Team. *What is Angular? - Official Documentation*. 2025. URL: https://angular.dev/overview (cit. on pp. 17, 18).

[20] Besant Technologies. *What is Angular?* URL: https://www.besanttechnologies.com/what-is-angular (cit. on p. 18).

[21] ReactiveX. *ReactiveX: The Observer pattern done right*. 2025. URL: https://reactivex.io/ (cit. on p. 18).

[22] Zoumana Keita. *NoSQL Databases: What Every Data Scientist Needs to Know*. 2023. URL: https://www.datacamp.com/blog/nosql-databases-what-every-data-scientist-needs-to-know (cit. on p. 19).

[23] Roshni Verma. *MongoDB Architecture: A Comprehensive Guide*. 2024. URL: https://medium.com/@roshniverma021/mongodb-architecture-a-comprehensive-guide-c72799347b22 (cit. on pp. 19, 20, 22).

[24] Victoria Malaya. *SQL vs. NoSQL*. 2013. URL: https://sql-vs-nosql.blogspot.com/2013/11/indexes-comparison-mongodb-vs-mssqlserver.html (cit. on p. 21).

[25] MongoDB Inc. *Aggregation Operations*. 2025. URL: https://www.mongodb.com/docs/manual/aggregation/ (cit. on p. 21).

[26] MongoDB Inc. *Transactions*. 2025. URL: https://www.mongodb.com/docs/manual/core/transactions/ (cit. on p. 22).

[27] Amazon Web Services (AWS). *What is a RESTful API?* 2025. URL: https://aws.amazon.com/what-is/restful-api/ (cit. on pp. 23, 24).

[28] IBM. *GraphQL vs. REST: What's the Difference?* 2023. URL: https://www.ibm.com/topics/graphql-vs-rest (cit. on pp. 24, 25).

[29] Apollo GraphQL. *Introduction to GraphQL*. 2024. URL: https://www.apollographql.com/docs/intro/ (cit. on pp. 25, 27).

[30] Amazon Web Services (AWS). *What is DevOps?* 2025. URL: https://aws.amazon.com/devops/what-is-devops/ (cit. on pp. 27, 28).

[31] Atlassian. *DevOps: A complete guide*. 2024. URL: https://www.atlassian.com/devops (cit. on p. 28).

[32] Red Hat. *What is CI/CD?* 2023. URL: https://www.redhat.com/en/topics/devops/what-is-ci-cd (cit. on pp. 28, 29).

[33] GitLab. *What is CI/CD? Continuous integration and continuous delivery explained*. 2025. URL: https://about.gitlab.com/topics/ci-cd/ (cit. on p. 29).

[34] Microsoft Learn. *Backends for Frontends pattern*. 2024. URL: https://learn.microsoft.com/en-us/azure/architecture/patterns/backends-for-frontends (cit. on p. 35).

[35] Sam Newman. *Pattern: Backends For Frontends*. 2015. URL: https://samnewman.io/patterns/architectural/bff/ (cit. on p. 35).

[36] Michael Szczepanik. *Backend for frontend (BFF) pattern — why do you need to know it?* 2021. URL: https://medium.com/mobilepeople/backend-for-frontend-pattern-why-you-need-to-know-it-46f94ce420b0 (cit. on p. 36).